🔵 PHASE 1 — NUMPY BASICS (Deep Mastery Notes)

You can directly paste this into Word.

1️⃣ Why NumPy Is Faster Than Python Lists

NumPy is faster because of **three major architectural advantages**:

1.1 Homogeneous Data Storage

Python lists can store mixed data types:

```
[1, "hello", 3.14]
```

NumPy arrays store only one data type:

```
np.array([1, 2, 3])
```

Why This Matters

- All elements have **same size in memory**
- No need to check type of each element during computation
- CPU performs operations directly

This removes runtime overhead.

1.2 Contiguous Memory Block

Python list:

- Stores references (pointers) to objects
- Objects scattered in memory

NumPy array:

- Stores actual values

- Stored in one continuous block of memory

Why This Matters

- Better CPU cache usage
- Faster memory access
- No pointer jumping

This drastically improves performance.

1.3 Vectorized Operations (C-Level Execution)

Python list:

```
for i in range(len(list)):
    list[i] *= 2
```

Executed in Python loop (slow).

NumPy:

```
arr * 2
```

Executed in optimized C code internally.

Why This Matters

- No Python loop
- No interpreter overhead
- Uses low-level optimized libraries (BLAS, SIMD)

This is the biggest speed advantage.

2 The ndarray Object (Core of NumPy)

`ndarray` = N-dimensional array.

It is NOT just data.

It consists of:

1. Data buffer (actual numbers in memory)
2. Metadata (information about how to interpret data)

2.1 What ndarray Stores

A. Data Buffer

- Raw binary data
- Stored contiguously

B. Metadata

- shape
- ndim
- dtype
- strides
- itemsize

The metadata tells NumPy how to interpret raw memory.

3 Important ndarray Attributes

3.1 shape

Tuple showing dimensions of array.

Example:

```
np.array([[1,2,3],[4,5,6]])
```

Shape = (2, 3)

Meaning:
2 rows, 3 columns

3.2 ndim

Number of dimensions.

Examples:

- [1,2,3] → ndim = 1

- [[1,2],[3,4]] → ndim = 2

3.3 size

Total number of elements.

Example:
(2 × 3) = 6

3.4 dtype

Specifies data type of elements.

Examples:

- int32
- int64
- float32
- float64

Why dtype Matters

It determines:

- Memory usage
- Speed
- Precision

3.5 itemsize

Size (in bytes) of one element.

Examples:

- int8 → 1 byte
- int32 → 4 bytes
- float64 → 8 bytes

3.6 nbytes

Total memory consumed by array.

Formula:

```
nbytes = size × itemsize
```

Example:
If 6 elements and each is 4 bytes:
6 × 4 = 24 bytes

4️⃣ Memory Model Basics

Understanding this makes you advanced.

4.1 Contiguous Memory Layout

NumPy stores elements in row-major order (C-style by default).

Example:

```
[[1,2,3],
 [4,5,6]]
```

Stored in memory as:

```
1 2 3 4 5 6
```

Single continuous block.

4.2 Strides

Strides tell:

How many bytes to jump in memory to move to next element in each dimension.

Example:
If dtype = int32 (4 bytes)

For 2×3 array:
Strides might be:
(12, 4)

Meaning:

- Move 12 bytes to go to next row
- Move 4 bytes to go to next column

4.3 flags

Tells memory properties:

- C_CONTIGUOUS (row-major)
- F_CONTIGUOUS (column-major)
- OWNDATA (does it own memory?)

5️⃣ Copy vs View (Very Important)

5.1 View

A view does NOT copy data.

It only changes how data is interpreted.

Example:

```
slice_arr = arr[2:7]
```

Modifying slice modifies original.

Because both share same memory buffer.

5.2 Copy

Creates independent memory.

Example:

```
new_arr = arr.copy()
```

Now modifying new_arr does NOT affect original.

5.3 reshape and Memory

`reshape()` usually returns a view if possible.

If memory layout allows → no copy
If not → creates copy

6️⃣ Why Slicing Returns Views

Because slicing only adjusts:

- Starting pointer
- Shape
- Strides

It does NOT allocate new memory.

This is extremely efficient.

7️⃣ Memory Calculation Logic

Example:

```
a = np.array([1,2,3], dtype=np.int8)
```

int8 = 1 byte
3 elements

Total memory:
3 × 1 = 3 bytes

Always use:

```
size × itemsize
```

🔥 Final Mental Model

Think of ndarray as:

Raw memory block + instruction manual

The raw memory is the data.
The instruction manual is metadata (shape, strides, dtype).

Change metadata → view.
Change data → affects all views.

📌 Summary of What You Must Remember

- NumPy is fast because of contiguous memory + homogeneous dtype + C-level vectorization.
- ndarray = data buffer + metadata.
- size × itemsize = memory used.
- Slicing returns view.
- reshape usually returns view.
- Strides control memory jumping.
- dtype controls memory and precision.